



MODELLI E ARCHITETTURE AVANZATI DI BASI DI DATI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Indice

I	Basi di Dati - Prof. Sapino	3
1	Introduzione alle Basi di Dati e ai Modelli Storici	3
1.1	Concetti Fondamentali	3
1.2	Livelli di Astrazione (Architettura Standard)	3
1.3	DBMS (Data Base Management System)	4
2	Carrellata Storica dei Modelli di Dati	5
2.1	Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)	5
2.2	Seconda Generazione: Modello Relazionale (Anni '70)	5
2.3	Terza Generazione: Anni '80 e il Modello ad Oggetti	6
2.4	Quarta Generazione: Anni '90 e 2000 - Dati Semistrutturati e Grafi	6
2.5	Modelli Ad-Hoc per Domini Specifici	7
II	Anselma	8
3	Data Intensive Applications	8
3.1	Building Blocks	8
3.2	Ruoli e Professioni nei Dati	8
3.3	Sistemi Operazionali vs Analitici	9
3.3.1	OLTP (Online Transaction Processing)	9
3.3.2	OLAP (Online Analytical Processing)	9
3.3.3	Confronto Sintetico	9
3.4	Data Warehouse	9
3.4.1	Il Processo ETL	10
3.5	Data Lake	10
3.5.1	Definizione e Principio Sushi	10
3.6	Cloud vs Self-Hosting	11
3.6.1	Self-Hosting (On-Premises)	11
3.6.2	Cloud Computing	11
3.7	Distributed Versus Single-Node Systems	11
3.7.1	Motivazioni per la Distribuzione	11
3.7.2	Sfide e Complessità	12
3.8	Data Systems, Law, and Society	12
3.9	Esempio Pratico: Tech Stack di Netflix	12
3.10	Case Study: Social Network Home Timelines	13
3.10.1	Requisiti e Carico di Lavoro	13

3.10.2	Approccio 1: Pull (Relazionale)	13
3.10.3	Approccio 2: Push (Materializzazione)	14
3.10.4	Approccio 3: Ibrido	14
3.11	Reliability, Scalability, Maintainability	14
3.11.1	Reliability (Affidabilità)	15
3.11.2	Scalability (Scalabilità)	15
3.11.3	Maintainability (Manutenibilità)	16

Basi di Dati - Prof. Sapino

1 Introduzione alle Basi di Dati e ai Modelli Storici

1.1 Concetti Fondamentali

Database (Base di Dati) Un database è una collezione di dati organizzati in modo strutturato, in funzione del modello di riferimento scelto. È fondamentale che i dati siano organizzati in modo efficiente per poterne garantire una gestione e un'interrogazione ottimale (reperimento, analisi e visualizzazione univoca dei risultati).

Differenza tra Dato e Informazione

- **Dato:** È un elemento grezzo, privo di contesto (es. un numero, una stringa, una data).
- **Informazione:** È il dato interpretato e contestualizzato.

Esempio: Il numero "10" è un dato. Se lo associo al numero di studenti presenti a lezione, diventa un'informazione. Se lo associo al numero di libri che ho letto, diventa un'informazione diversa. Il dato di partenza è lo stesso, ma il contesto ne cambia il significato. Le basi di dati organizzano i dati proprio per rendere efficiente e rapida l'estrazione di queste informazioni.

Modello dei Dati È un formalismo matematico e rigoroso utilizzato per descrivere la struttura dei dati stessi (es. modello relazionale, a oggetti, a grafo). Ciascun modello esprime un insieme di vincoli intrinseci che l'organizzazione dei dati deve rispettare. Nel modello relazionale, ad esempio, i dati devono essere divisi in tabelle normalizzate, collegate tramite chiavi primarie ed esterne.

Schema Rappresenta l'applicazione di un modello di dati a un dominio specifico. Definisce un insieme di vincoli strutturali e di integrità. In un database relazionale, lo schema descrive le tabelle (relazioni), i campi (attributi), i tipi di dato e i vincoli (es. chiavi primarie ed esterne).

I vincoli sono essenziali per garantire l'**integrità dei dati** (es. evitare problemi di integrità referenziale se una chiave esterna punta a un record inesistente) e semplificano l'ottimizzazione e la gestione, sebbene limitino la flessibilità del modello per certi scopi.

1.2 Livelli di Astrazione (Architettura Standard)

L'organizzazione dei dati può essere analizzata su tre livelli principali:

1. **Livello Concettuale:** È il livello più alto e astratto. Descrive "cosa" deve essere rappresentato (la conoscenza del dominio) senza preoccuparsi dell'implementazione. Utilizza formalismi comprensibili agli utenti, come i diagrammi ER (Entità-Relazione) o UML.
2. **Livello Logico:** Si posiziona a metà strada e colma il gap tra concettuale e fisico. Descrive i dati in modo formale attraverso le strutture previste dal modello scelto (es. tabelle nel modello relazionale o gerarchie nel modello ad oggetti), ma rimane indipendente dalla loro effettiva memorizzazione fisica. Alcuni modelli logici (come quello ad oggetti) sono più vicini al concettuale, altri (come il relazionale) sono più vicini al livello fisico.
3. **Livello Fisico:** Descrive "come" i dati sono effettivamente memorizzati su disco. Riguarda le strutture dati interne, l'allocazione dello spazio, la codifica in bit (es. come rappresentare fisicamente una data) e gli indici.

1.3 DBMS (Data Base Management System)

Cos'è un DBMS È un sistema software (che talvolta include componenti hardware) progettato per creare, gestire, manipolare e interrogare una base di dati in modo efficiente e sicuro. L'obiettivo principale di un DBMS è garantire la separazione tra la "business logic" (le applicazioni) e i dati fisici. Per fare ciò, deve garantire due livelli di astrazione:

- **Indipendenza Logica dei Dati:** Le applicazioni non devono risentire di modifiche fatte allo schema logico (es. se aggiungo una colonna a una tabella, un programma che interrogava le altre colonne deve continuare a funzionare senza essere riscritto).
- **Indipendenza Fisica dei Dati:** Le applicazioni e lo schema logico non devono risentire di modifiche a livello fisico (es. se aggiungo o rimuovo un indice su un disco per velocizzare una ricerca, il programma esterno non se ne deve accorgere).

Differenza e Interazione tra OS e DBMS Un Sistema Operativo (OS) offre funzionalità di base per la gestione delle risorse hardware e software, come la gestione dei file, della memoria centrale (buffer), della concorrenza e il recupero da malfunzionamenti (crash recovery). Sebbene un DBMS si appoggi all'OS per l'accesso fisico alle risorse, esso implementa versioni *ad hoc* e altamente specializzate di queste funzionalità. I meccanismi generici dell'OS risultano infatti inadeguati per le esigenze specifiche di una base di dati.

Le differenze principali si manifestano in quattro macro-aree:

- **Gestione del Buffer (Buffer Management):** Nei database, l'operazione di gran lunga più costosa è l'I/O su disco. Entrambi i sistemi utilizzano un buffer in RAM per minimizzare le letture/scritture fisiche, ma adottano strategie di *replacement* profondamente diverse per scegliere la "vittima" (la pagina di memoria da sovrascrivere quando la RAM è piena). L'OS utilizza politiche generiche e reattive, come l'LRU (Least Recently Used), che scarta la pagina non utilizzata da più tempo. Questa politica può essere disastrosa per un DBMS: ad esempio, durante la scansione sequenziale di una tabella molto grande, l'LRU svuoterebbe l'intero buffer scartando dati potenzialmente utili per altre query. Il DBMS, invece, ha un comportamento **predittivo**. Grazie al modulo di ottimizzazione delle query (Query Optimizer), il DBMS conosce in anticipo il piano di esecuzione e sa quali dati serviranno a breve, potendo così forzare il mantenimento in memoria di pagine strategiche o scartare quelle inutili.
- **Gestione dei File e Strutture Dati:** Per l'OS, un file è semplicemente una sequenza (stream) di byte non strutturata. Per il DBMS, i dati hanno una struttura semantica precisa: file fisici suddivisi in blocchi, contenenti tuple, formate da attributi. Spesso il DBMS bypassa del tutto le routine di caching dei file dell'OS per avere il controllo assoluto su *quando* un blocco viene effettivamente scritto su disco.
- **Gestione della Concorrenza (Concurrency Control):** L'OS gestisce la concorrenza a livello di processi e thread. Il DBMS deve invece gestire la concorrenza a **livello logico del dato** (es. applicando lock temporanei su una singola riga di una tabella). Questo permette a migliaia di utenti simultanei di leggere e aggiornare gli stessi dati senza creare inconsistenze.
- **Recupero dai Guasti (Crash Recovery):** Mentre l'OS preserva la coerenza del file system, il DBMS deve garantire a livello logico l'integrità dei dati e il rispetto delle proprietà ACID. Tramite file di log e protocolli come il **Write-Ahead Logging (WAL)**, il DBMS può eseguire l'UNDO (annullare transazioni a metà) e il REDO (riapplicare transazioni completate) dopo un crash.

Transazione È una sequenza di operazioni trattata come un'unità logica e **atomica**: o tutte le operazioni vengono eseguite con successo, o nessuna viene applicata (principio di "tutto o niente"). *Esempio:* In un bonifico bancario (prelievo da un conto e deposito in un altro), se il deposito fallisce, il prelievo deve essere annullato per mantenere la consistenza ed evitare la perdita di denaro.

2 Carrellata Storica dei Modelli di Dati

2.1 Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)

Modello Gerarchico Basato su una struttura ad albero. La radice rappresenta il concetto principale e i nodi figli le sue proprietà o entità dipendenti (es. Radice: "Persona" → Figli: "Nome", "Data di Nascita"). I puntatori erano bidirezionali (padre-figlio e figlio-padre).

- **Pro:** Molto efficiente per relazioni strettamente gerarchiche.
- **Contro:** Estremamente limitante per relazioni complesse o molti-a-molti, dove un nodo dovrebbe avere "più padri".

Modello Reticolare (Codasyl) Estende il modello gerarchico trasformando l'albero in un grafo. Questo permetteva ai nodi di avere molteplici nodi padre e di esprimere le relazioni tramite archi.

Criticità della Prima Generazione

1. **Distanza dal modello concettuale:** Erano funzionali solo per app di natura procedurale, ma inadeguati per sistemi come ospedali o banche.
2. **Navigazione a Puntatori Fisici:** Si sapeva da dove partiva il puntatore ma non dove andava a finire nel disco fisicamente.
3. **Linguaggio fortemente Procedurale:** L'utente doveva essere un bravo programmatore, scrivendo la procedura esatta (il percorso dei puntatori) per raggiungere il dato (es. "Vai a Università → segui puntatore Studenti...").
4. **Imprevedibilità e scarsa ottimizzazione:** L'ottimizzatore non poteva prevedere il percorso di accesso (e quindi allocare la memoria o posizionare la testina del disco in modo efficiente), rendendo impossibile un'ottimizzazione automatica.

2.2 Seconda Generazione: Modello Relazionale (Anni '70)

Introdotta da E.F. Codd, è il modello tuttora più diffuso. I dati sono organizzati in **frammenti omogenei** (le relazioni, o tabelle). Ciascun record (riga) ha la stessa struttura formale degli altri.

Vantaggi principali:

- **Prevedibilità e Ottimizzazione:** L'omogeneità permette di stimare quante pagine di memoria bisognerà leggere. "Prevedibilità implica ottimizzabilità" (si sfruttano al meglio i movimenti fisici del disco).
- **Assenza di Puntatori:** Si usano **puntatori logici** (le chiavi esterne per fare JOIN) eliminando l'imprevedibilità del puntatore fisico.
- **Linguaggio Dichiarativo (SQL):** L'utente dice *cosa* vuole estrarre, e non *come* trovarlo. Sarà il DBMS a tradurlo nei passi fisici necessari.
- **Algebra Relazionale:** Bastano pochi operatori matematici per implementare le richieste SQL. Attenzione: l'SQL è dichiarativo, mentre l'algebra relazionale è procedurale. Dato che gli operatori sono pochi, ci sono meno combinazioni possibili per ottimizzare (anche se l'ottimizzazione resta un problema matematico NP-completo).
- **Vicinanza al Modello Fisico:** Le righe piatte nelle tabelle sono facilmente mappabili in blocchi contigui su disco.

Svantaggi e Limiti del Modello Relazionale:

- **Dati Piatti ("Particellati"):** Per ricomporre un'informazione complessa bisogna eseguire costosi e numerosi JOIN.
- **Difficoltà con Strutture Complesse:** Non è possibile rappresentare nativamente l'ereditarietà (es. Persona → Studente/Docente) o gerarchie complesse.
- **Incapacità di Match Parziali:** Il relazionale restituisce risultati esatti. Se cerco dati per "somiglianza" (match parziali), il relazionale fallisce perché questa logica andrebbe implementata nell'applicativo esterno, violando il principio di indipendenza dei dati.

2.3 Terza Generazione: Anni '80 e il Modello ad Oggetti

Per superare i limiti del relazionale (soprattutto riguardo all'ereditarietà), si sono sviluppate due scuole di pensiero, entrambe basate sul concetto di *Oggetto*:

- **Object-Oriented (OO) - Spinta dall'Accademia:** Si abbandona totalmente il relazionale. Si creano basi di dati che memorizzano nativamente non solo i dati, ma anche i metodi (comportamenti). Nativamente supporta gerarchie ed ereditarietà. Si interrogano con linguaggi come l'OQL (Object Query Language).
- **Object-Relational (OR) - Spinta dall'Industria:** Abbandonare il relazionale era economicamente folle visto il suo successo. Nasce così un approccio "ibrido": un motore relazionale di base arricchito da funzionalità ad oggetti utilizzabili solo "se necessarie", mantenendo la compatibilità con i vecchi sistemi.

2.4 Quarta Generazione: Anni '90 e 2000 - Dati Semistruutturati e Grafi

Con l'avvento di Internet, i database non sono più "isole" chiuse in una singola azienda, ma devono comunicare. Scambiarsi tabelle relazionali era problematico per colpa degli schemi rigidi.

Basi di Dati Semistructurate (XML) Si cercava flessibilità (es. in un DB l'indirizzo è un campo unico, in un altro è diviso in via/civico). Nasce l'esigenza di documenti **autodescrittivi** come l'XML. In questi sistemi, **lo schema stesso diventa interrogabile**. *Esempio:* "Dimmi se esiste il campo 'Indirizzo' in questo nodo, e se sì, dammi il suo valore". Anche qui vinse un approccio ibrido: esistono DB nativamente XML, ma i giganti dell'industria hanno integrato il supporto XML sopra i propri DB Relazionali.

Database a Grafo (dal 2003) A differenza degli anni '60 in cui si forzava la struttura a grafo per tutto, ora i grafi vengono usati *esclusivamente* per domini in cui i concetti sono naturalmente legati da complesse reti di relazioni.

- **Web Semantico e RDF:** Permette di descrivere la conoscenza tramite **triplette (Soggetto, Predicato, Oggetto)**. Lo schema non è rigido né definito a priori (es. "Sapino" → gestisce → "Corso"). Posso effettuare **query di raggiungibilità** che nei modelli precedenti erano impossibili (es. "Trova il percorso relazionale più breve che collega il prof. X al dipartimento Y"). Si usa il linguaggio dichiarativo SPARQL.
- **Social Network:** Simili all'RDF ma con milioni di nodi (persone) e poche tipologie di archi (es. "Follow", "Like"). Se lo facessimo nel modello relazionale servirebbero infiniti JOIN. I DB a grafo permettono di calcolare facilmente il ruolo di un nodo (es. la "centralità" di un influencer) o di ricercare pattern ricorrenti (usati moltissimo ad esempio in biologia per studiare le catene di proteine).

2.5 Modelli Ad-Hoc per Domini Specifici

Oggi si è compreso che un "modello universale" non basta per tutto, e sono nati sistemi specializzati:

- **Database Spaziali (GIS):** L'elemento chiave è la *Location* (coordinate spaziali). Si usano per query impossibili in SQL tradizionale, come le *query di range / Nearest Neighbours* ("Trova i ristoranti entro 5km da me") e il **Join Spaziale** ("Abbina le scuole del comune con i supermercati più vicini a ciascuna di esse").
- **Database Documentali e Multimediali:** Permettono query basate sulla **somiglianza** e non sull'esatta uguaglianza (es. "Trova profili di lavoratori simili al mio").
- **Gestione dell'Incertezza:** I DB relazionali operano sotto l'assunzione di mondo chiuso (tutto ciò che è nel DB è vero al 100%, ciò che non c'è è falso). Alcuni modelli più moderni permettono di associare pesi di probabilità (es. "Lo studente X frequenta il corso Y con l'80% di confidenza").

Parte II

Anselma

3 Data Intensive Applications

Oggi, molte applicazioni software sono classificate come **data-intensive** (ad alta intensità di dati), in contrapposizione a quelle **compute-intensive** (ad alta intensità di calcolo).

Data-intensive: Sono applicazioni in cui il collo di bottiglia non è la potenza della CPU, ma la gestione dei dati stessi. Le sfide principali riguardano:

- La **mole** dei dati (volume);
- La **complessità** dei dati;
- La **velocità** con cui i dati cambiano.

Queste applicazioni devono garantire consistenza, disponibilità e affidabilità anche in presenza di guasti. Esempi tipici sono i social network, piattaforme di e-commerce, sistemi bancari e piattaforme di telemetria IoT.

Compute-intensive: Sono applicazioni dove il collo di bottiglia è la pura potenza di elaborazione (CPU). In questi casi, il volume di dati può essere ridotto, ma le operazioni matematiche o logiche richieste sono estremamente complesse (es. rendering video, simulazioni scientifiche).

3.1 Building Blocks

La maggior parte delle applicazioni data-intensive non è monolitica, ma è costruita combinando diverse tecnologie specializzate:

- **Database:** Per immagazzinare e interrogare i dati in modo persistente.
- **Cache:** Per memorizzare risultati costosi o dati acceduti frequentemente in memoria volatile, velocizzando le letture.
- **Search Indexes:** Per consentire ricerche complesse (es. per parole chiave o filtri) che un database tradizionale gestirebbe lentamente.
- **Stream Processing:** Per l'elaborazione asincrona di messaggi e eventi in tempo reale.
- **Batch Processing:** Per elaborare periodicamente grandi volumi di dati accumulati nel tempo.

3.2 Ruoli e Professioni nei Dati

La gestione di sistemi complessi richiede diverse figure professionali:

- **Backend Engineers:** Costruiscono i servizi che gestiscono le richieste operative (lettura/scrittura) dell'applicazione.
- **Business Analysts:** Utilizzano i dati per generare report e metriche di *Business Intelligence* (BI) a supporto delle decisioni manageriali.
- **Data Scientists:** Esplorano i dati per trovare pattern o creare modelli di Machine Learning (es. sistemi di raccomandazione "chi ha comprato X ha comprato anche Y").
- **Data Engineers:** Sono responsabili dell'infrastruttura dati; integrano i sistemi operazionali con quelli analitici e gestiscono le pipeline di dati.
- **Analytics Engineers:** Curano la trasformazione e la modellazione dei dati per renderli puliti e utilizzabili da analisti e data scientist.

3.3 Sistemi Operazionali vs Analitici

Esiste una distinzione fondamentale tra i sistemi utilizzati per gestire l'operatività quotidiana e quelli usati per l'analisi dei dati.

3.3.1 OLTP (Online Transaction Processing)

Sono i sistemi che supportano l'operatività dell'applicazione (es. il database di un e-commerce).

- **Scopo:** Gestire transazioni rapide e frequenti eseguite dagli utenti.
- **Accesso:** Letture e scritture di piccoli record identificati da una chiave.
- **Stato dei dati:** Rappresentano lo stato corrente (es. "qual è il saldo attuale?").

3.3.2 OLAP (Online Analytical Processing)

Sono sistemi ottimizzati per l'analisi dei dati da parte di analisti e data scientist.

- **Scopo:** Rispondere a domande complesse sul business (es. "Ricavi totali per mese").
- **Accesso:** Aggregazioni su grandi quantità di record (scansione di molte righe, poche colonne).
- **Stato dei dati:** Rappresentano la storia degli eventi nel tempo.

3.3.3 Confronto Sintetico

Proprietà	OLTP (Operazionale)	OLAP (Analitico)
Pattern di Lettura	Query puntuali (cerca per chiave), pochi record.	Aggregazioni su grandi moli di dati.
Pattern di Scrittura	Accesso casuale, aggiornamenti utente in tempo reale.	Importazione massiva (Batch/-Stream).
Utenti	Clienti, Applicazioni Backend.	Analisti, Data Scientists, Manager.
Dati	Stato corrente, dati normalizzati.	Storico degli eventi, dati denormalizzati.
Dimensione	Gigabyte - Terabyte.	Terabyte - Petabyte.

Tabella 1: Differenze tra OLTP e OLAP

3.4 Data Warehouse

Un **Data Warehouse** (DW) è un database separato, ottimizzato per carichi di lavoro OLAP. L'utilizzo diretto dei database OLTP per l'analisi è sconsigliato per tre motivi:

1. **Isolamento delle performance:** Le query analitiche sono pesanti e potrebbero rallentare l'applicazione per gli utenti finali.
2. **Data Silos:** I dati sono spesso sparsi su più sistemi operazionali incompatibili tra loro.
3. **Schema:** Gli schemi OLTP sono normalizzati (ottimi per la coerenza in scrittura), mentre l'analisi richiede schemi denormalizzati (ottimi per la velocità di lettura).

All'interno di un Data Warehouse, i dati devono essere non normalizzati per evitare costose operazioni di JOIN durante l'analisi. La denormalizzazione è una pratica comune in OLAP, dove si accettano dati ridondanti per ottenere prestazioni migliori nelle query analitiche.

Normalizzazione è un processo di progettazione del database che mira a ridurre la ridondanza dei dati e migliorare l'integrità dei dati. In un database normalizzato, i dati sono organizzati in tabelle separate con relazioni ben definite, e le operazioni di lettura richiedono spesso JOIN tra queste tabelle.

3.4.1 Il Processo ETL

Per popolare un Data Warehouse si utilizza il processo **ETL**:

- **Extract (E)**: Estrazione dei dati dai vari sistemi sorgente (OLTP).
- **Transform (T)**: Pulizia e trasformazione dei dati. In questa fase avviene spesso la *denormalizzazione*, convertendo i dati in schemi a stella o a fiocco di neve (Star/Snowflake schema) per evitare costose operazioni di JOIN durante l'analisi.
- **Load (L)**: Caricamento dei dati nel Data Warehouse.

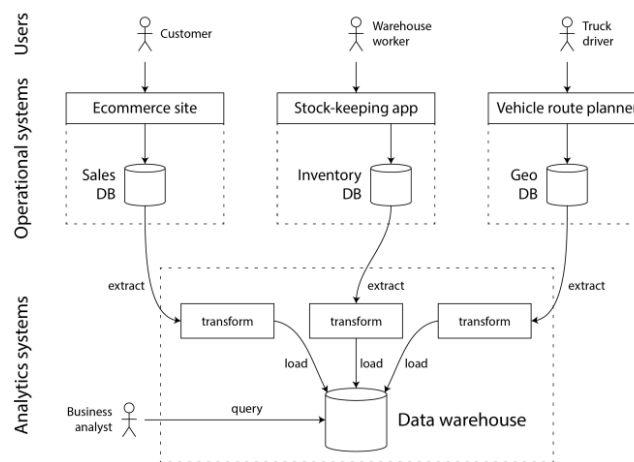


Figura 1: Schema del processo ETL: Dai sistemi operazionali al Data Warehouse

Varianti:

- **ELT**: In sistemi moderni potenti, si preferisce caricare i dati grezzi e trasformarli successivamente all'interno del Warehouse stesso.
- **CDC (Change Data Capture)**: Metodo di estrazione continua dove ogni modifica nel DB sorgente viene replicata quasi in tempo reale nel sistema di destinazione.

3.5 Data Lake

Il **Data Warehouse**, basandosi sul modello relazionale (SQL), ha dei limiti per le moderne applicazioni di Data Science e Machine Learning. I modelli di ML lavorano su vettori e matrici (features numeriche) e spesso richiedono l'analisi di dati **non strutturati** (testo, immagini, video, audio) che non si adattano alle rigide tabelle di un database.

3.5.1 Definizione e Principio Sushi

Un **Data Lake** è un repository centralizzato che contiene una copia di tutti i dati in formato **grezzo** (raw).

- **Contenuto**: File grezzi (CSV, JSON, immagini) o formati ottimizzati per i Big Data (Avro, Parquet).
- **Schema-on-read**: A differenza del Warehouse (Schema-on-write), nel Lake non si impone una struttura al momento del salvataggio. La struttura viene definita solo quando i dati vengono letti.

Il Principio Sushi: Il concetto alla base del Data Lake è che *"il crudo è meglio del cotto"*. Mantenere i dati grezzi offre la massima flessibilità: i dati possono essere trasformati in modi diversi in futuro per rispondere a domande che oggi non siamo ancora in grado di formulare, senza il rischio di aver perso informazioni durante una trasformazione prematura.

3.6 Cloud vs Self-Hosting

La scelta tra self-hosting e cloud dipende dalla gestione dell'infrastruttura e dalla variabilità del carico di lavoro.

3.6.1 Self-Hosting (On-Premises)

Il sistema è costruito su risorse di proprietà: sistema operativo general-purpose (Linux, Windows), filesystem, networking TCP/IP e hardware fisico (CPU, RAM) .

3.6.2 Cloud Computing

I servizi cloud sono particolarmente preziosi quando il carico sul sistema varia significativamente nel tempo. I modelli di servizio si differenziano per il livello di gestione affidato al vendor :

- **IaaS (Infrastructure as a Service):** Il vendor fornisce macchine virtuali (hardware virtuale). L'utente gestisce il sistema operativo, il middleware e le applicazioni .
- **PaaS (Platform as a Service):** Fornisce macchine virtuali e strumenti software per lo sviluppo. Il vendor gestisce anche il sistema operativo e il runtime .
- **SaaS (Software as a Service):** Il vendor fornisce un'applicazione pronta all'uso; l'utente gestisce solo l'accesso e l'utilizzo .
- **FaaS (Function as a Service / Serverless):** Consente di eseguire funzioni specifiche (es. transazioni di pagamento) senza gestire alcuna infrastruttura.

Esempi di sistemi:

Categoria	Self-hosted	Cloud-native
Operational (OLTP)	MySQL, PostgreSQL, MongoDB	Azure SQL, Google Spanner, AWS Aurora
Analytical (OLAP)	Teradata, ClickHouse, Spark	Snowflake, BigQuery, Azure Synapse

Tabella 2: Esempi di sistemi self-hosted e cloud-native per OLTP e OLAP

3.7 Distributed Versus Single-Node Systems

I **sistemi distribuiti** sono composti da più nodi che comunicano via rete.

3.7.1 Motivazioni per la Distribuzione

- **Tolleranza ai guasti:** La ridondanza permette al sistema di funzionare anche in caso di guasto a macchine o datacenter.
- **Scalabilità ed Elasticità:** Possibilità di gestire carichi superiori alla capacità di una singola macchina e scalare dinamicamente le risorse in base alla domanda (allineando costi e utilizzo).
- **Latenza:** Riduzione dei tempi di risposta tramite la distribuzione geografica dei servizi vicino agli utenti.

- **Hardware Specializzato:** Utilizzo di nodi ottimizzati per compiti specifici (es. GPU per Machine Learning, nodi ad alta densità di storage) .
- **Compliance e Sostenibilità:** Rispetto delle leggi sulla residenza dei dati (jurisdiction) e possibilità di schedare i carichi dove l'energia rinnovabile è più abbondante o economica .

3.7.2 Sfide e Complessità

L'architettura distribuita introduce criticità non presenti nei sistemi single-node:

- **Efficienza della comunicazione:** La comunicazione via rete è significativamente più lenta rispetto alle chiamate in-process. In alcuni casi, è più efficiente spostare la computazione verso la macchina che possiede già i dati.
- **Performance:** Un singolo sistema single-threaded potente può talvolta superare un cluster distribuito con oltre 100 core CPU.
- **Affidabilità della rete:** Incertezza sulla ricezione delle richieste e problemi di **idempotenza** (rischio di rieseguire operazioni non sicure in caso di retry) .
- **Observability:** La risoluzione dei problemi richiede strumenti avanzati di monitoraggio come log, metriche e tracing (es. OpenTelemetry).
- **Consistenza:** Mantenere i dati coerenti tra servizi diversi è intrinsecamente difficile.

3.8 Data Systems, Law, and Society

Le normative legali influenzano direttamente l'architettura tecnica:

- **Data Residency:** Le leggi possono imporre che i dati rimangano entro specifici confini nazionali.
- **Diritto alla cancellazione:** Questo richiede procedure tecniche rigorose per eliminare i dati non solo dai database attivi, ma anche dai sistemi di backup e storage a lungo termine, garantendo una cancellazione sicura e totale.

3.9 Esempio Pratico: Tech Stack di Netflix

Netflix utilizza un approccio ibrido e distribuito per gestire la complessità:

- **Database Relazionali:** MySQL per billing e sottoscrizioni; Amazon Aurora per consolidare l'infrastruttura SQL; CockroachDB per transazioni globali con consistenza multi-regione.
- **NoSQL:** Apache Cassandra per la cronologia di visione e raccomandazioni; EVCache (basato su Memcached) per il caching distribuito; Atlas per metriche operative time-series.
- **Data Lake:** Amazon S3 con Apache Iceberg per dati analitici e non strutturati (log, video metadata).

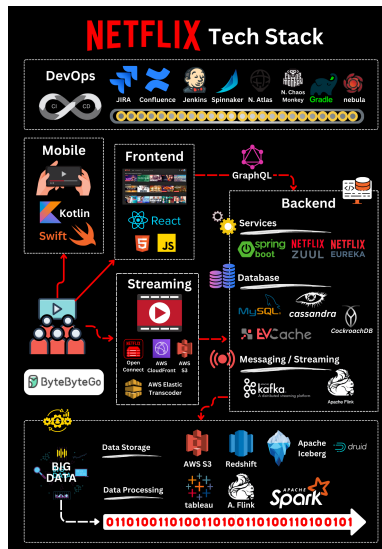


Figura 2: Esempio di Tech Stack distribuito e ibrido di Netflix

3.10 Case Study: Social Network Home Timelines

Prendiamo come esempio un social network come X (Twitter) per analizzare come i carichi di lavoro influenzino l'architettura.

3.10.1 Requisiti e Carico di Lavoro

I dati medi del sistema sono:

- **Post:** circa 5.700 post/secondo in media, con picchi che possono raggiungere i 150.000 post/secondo.
- **Consultazione Timeline:** gli utenti consultano la propria timeline circa 2 milioni di volte al secondo.
- **Connessioni:** un utente medio segue 200 persone e ha 200 follower, ma esistono casi estremi come celebrità con oltre 100 milioni di follower.

3.10.2 Approccio 1: Pull (Relazionale)

In questo approccio, i post vengono salvati in una tabella globale e la timeline viene generata "al volo" ogni volta che un utente la richiede. Per implementare la timeline della home, il database utilizza la tabella `follows` per trovare tutti gli utenti seguiti dall'utente corrente, recupera i post recenti di quegli utenti e li ordina per timestamp.

Query per la generazione della timeline (Pull)

```
SELECT posts.*, users.* FROM posts
  JOIN follows ON posts.sender_id = follows.followee_id
  JOIN users ON posts.sender_id = users.id WHERE follows.follower_id =
    $CURRENT_USER$
ORDER BY posts.timestamp DESC
LIMIT 1000
```

- **Efficienza in scrittura:** Inserire un nuovo post è economico poiché richiede solo un semplice inserimento nella collezione globale dei post.
- **Costo in lettura:** Caricare la timeline con questa query è estremamente costoso.

- **Impatto sul sistema:** Se un utente segue 200 persone, il sistema deve recuperare e unire le liste dei post di ciascuna di esse. Con 2 milioni di query al secondo, il database dovrebbe eseguire circa 400 milioni di lookup di post al secondo.
- **Scalabilità:** Per gli utenti che seguono decine di migliaia di account, questa query è difficilmente ottimizzabile e molto lenta da eseguire.

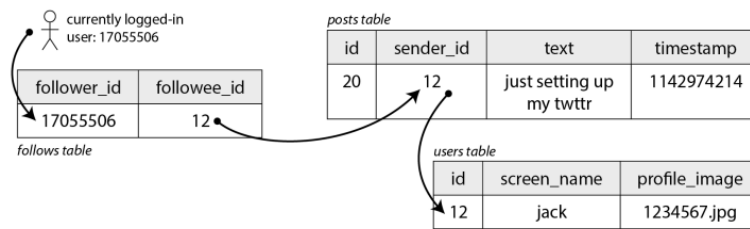


Figura 3: Semplice schema relazionale per l'implementazione di una timeline della home di X

3.10.3 Approccio 2: Push (Materializzazione)

Per ovviare ai costi di lettura, si possono precalcolare i risultati e servirli tramite una cache.

- **Materializzazione:** Consiste nel precalcolare e aggiornare i risultati di una query; la cache della timeline è un esempio di **vista materializzata**.
- **Fan-out:** Al momento della pubblicazione, il post viene consegnato alla cache di ogni follower.
- **Confronto prestazioni:** Con 5.700 post al secondo e 200 follower medi, si effettuano circa 1 milione di scritture al secondo, un risparmio significativo rispetto ai 400 milioni di lookup dell'approccio pull.
- **Problema degli Influencer:** Per gli utenti con milioni di follower (VIP), il lavoro di inserimento simultaneo in milioni di timeline diventa eccessivo.

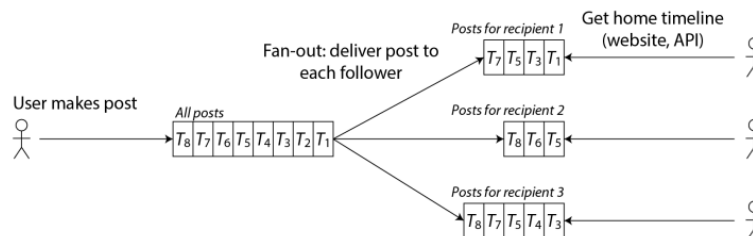


Figura 4: Consegnare i nuovi post a tutti i follower

3.10.4 Approccio 3: Ibrido

La soluzione più efficiente prevede un modello misto:

- Si usa la **materializzazione** (push) per gli utenti comuni.
- Si usa l'**approccio originale** (pull) per i post degli influencer, unendoli alla timeline materializzata solo al momento della lettura.

Lezione appresa: Non esiste un modello di dati perfetto in assoluto; è fondamentale scegliere il modello che meglio si allinea ai **pattern di accesso** ai dati del sistema specifico.

3.11 Reliability, Scalability, Maintainability

Nello sviluppo di sistemi software, e in particolare di applicazioni data-intensive, tre concetti sono di primaria importanza: l'affidabilità, la scalabilità e la manutenibilità.

3.11.1 Reliability (Affidabilità)

Un sistema è considerato affidabile se continua a funzionare correttamente anche in presenza di guasti o errori umani. È fondamentale distinguere tra:

- **Fault (Guasto):** Quando una singola componente del sistema devia dalle sue specifiche.
- **Failure (Fallimento):** Quando il sistema nel suo complesso smette di fornire il servizio all'utente.

L'obiettivo non è prevenire ogni singolo guasto (impossibile), ma progettare meccanismi di **fault-tolerance** per evitare che i guasti si trasformino in fallimenti.

Le principali fonti di guasto includono:

- **Guasti Hardware:** I guasti ai singoli dischi o macchine sono la norma su larga scala (es. in un cluster con 10.000 dischi, in media uno muore ogni giorno). La difesa consiste nella ridondanza (RAID, backup, alimentatori doppi) e in software capaci di tollerare la perdita di intere macchine.
- **Guasti Software:** Più difficili da prevedere rispetto all'hardware, sono spesso sistematici (es. bug nel kernel, memory leak, race conditions). Possono causare *fallimenti a cascata* nei sistemi distribuiti.
- **Errori Umani:** Gli studi indicano che la maggior parte dei periodi di inattività è causata da errori di configurazione o operativi degli esseri umani. Si combattono con buone UI, sandbox per i test e la capacità di effettuare rapidi **rollback** delle modifiche.

3.11.2 Scalability (Scalabilità)

La scalabilità è la capacità del sistema di gestire la crescita, sia essa in termini di volume di dati, carico di traffico o complessità delle operazioni.

Performance vs Scalabilità Mentre la **performance** indica quanto velocemente il sistema risponde con un carico fisso, la **scalabilità** descrive come il sistema si comporta all'aumentare del carico. Un sistema scalabile mantiene buone prestazioni anche quando il carico cresce.

Metriche di Prestazione

- **Response Time:** Il tempo totale visto dal client, inclusi ritardi di rete e code.
- **Service Time:** Il tempo effettivo durante il quale il servizio sta processando la richiesta.
- **Queueing Delays (Ritardi di Coda):** Possono verificarsi in diversi punti del flusso, ad esempio:
 - Attesa che una CPU si liberi per iniziare l'elaborazione.
 - Bufferizzazione dei pacchetti di risposta (es. TCP buffer) prima dell'invio se l'interfaccia di rete è congestionata da altri task.
- **Latency (Latenza):** Il tempo in cui una richiesta è "latente", ovvero non viene attivamente processata (es. ritardi in coda o in rete).
- **Throughput:** Numero di operazioni al secondo; metrica vitale per i sistemi *batch*.

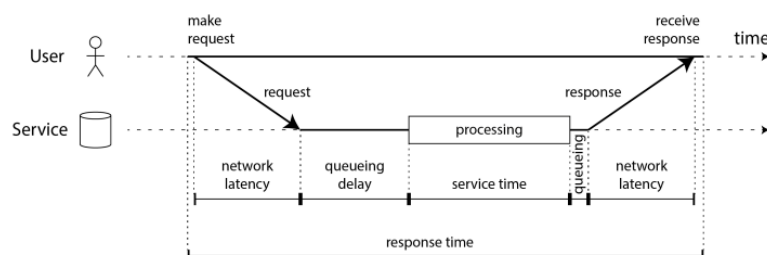


Figura 5: Diagramma la latenza e il tempo di risposta

Per misurare correttamente i tempi di risposta, non si usano le medie, ma i **percentili**:

$Xth\ percentile = Y\ ms$ significa che il $X\%$ delle richieste ha un tempo di risposta inferiore a $Y\ ms$

- **Mediana (p50):** Indica il tempo di risposta tipico per l'utente.
- **p95, p99 (Tail Latencies):** Rappresentano i casi peggiori. Ottimizzare il **p99.9** è cruciale per aziende come Amazon, poiché gli utenti con più dati (i più alto spendenti) sono spesso quelli che subiscono le latenze peggiori.

Esempio: L'impatto economico delle Tail Latencies (Amazon) Gli utenti che sperimentano le latenze peggiori (tail latencies) sono spesso i clienti più preziosi, poiché possiedono più dati e hanno effettuato più acquisti. Amazon ha osservato che:

- Un aumento di soli **100 ms** nel tempo di risposta riduce le vendite dell'**1%**.
- L'ottimizzazione del **99.99-esimo percentile** (la richiesta più lenta su 10.000) è stata ritenuta troppo costosa in termini di risorse rispetto al beneficio economico marginale, portando l'azienda a concentrarsi sul 99.9-esimo percentile.

Scaling Verticale vs Orizzontale

- **Verticale (Scaling Up):** Acquistare una macchina più potente. Semplice ma limitato dall'hardware.
- **Orizzontale (Scaling Out):** Aggiungere più macchine al cluster. Essenziale per sistemi web su larga scala.
- **Elasticità:** Capacità di alcuni sistemi cloud di aggiungere o rimuovere automaticamente risorse in base al carico corrente.

3.11.3 Maintainability (Manutenibilità)

La maggior parte del costo di un software risiede nella sua manutenzione continua, non nello sviluppo iniziale. Un sistema manutenibile deve essere facile da operare, comprendere e adattare.

I tre principi cardine del design sono:

1. **Operability (Operatività):** Rendere i compiti di routine semplici e sicuri per i team operativi.
2. **Simplicity (Semplicità):** Rimuovere la "complessità accidentale" per rendere il sistema comprensibile.
3. **Evolvability (Evolubilità):** Permettere modifiche sicure al sistema per adattarsi a nuovi requisiti senza riscritture massicce.